Agenda

1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

mySQL-Client

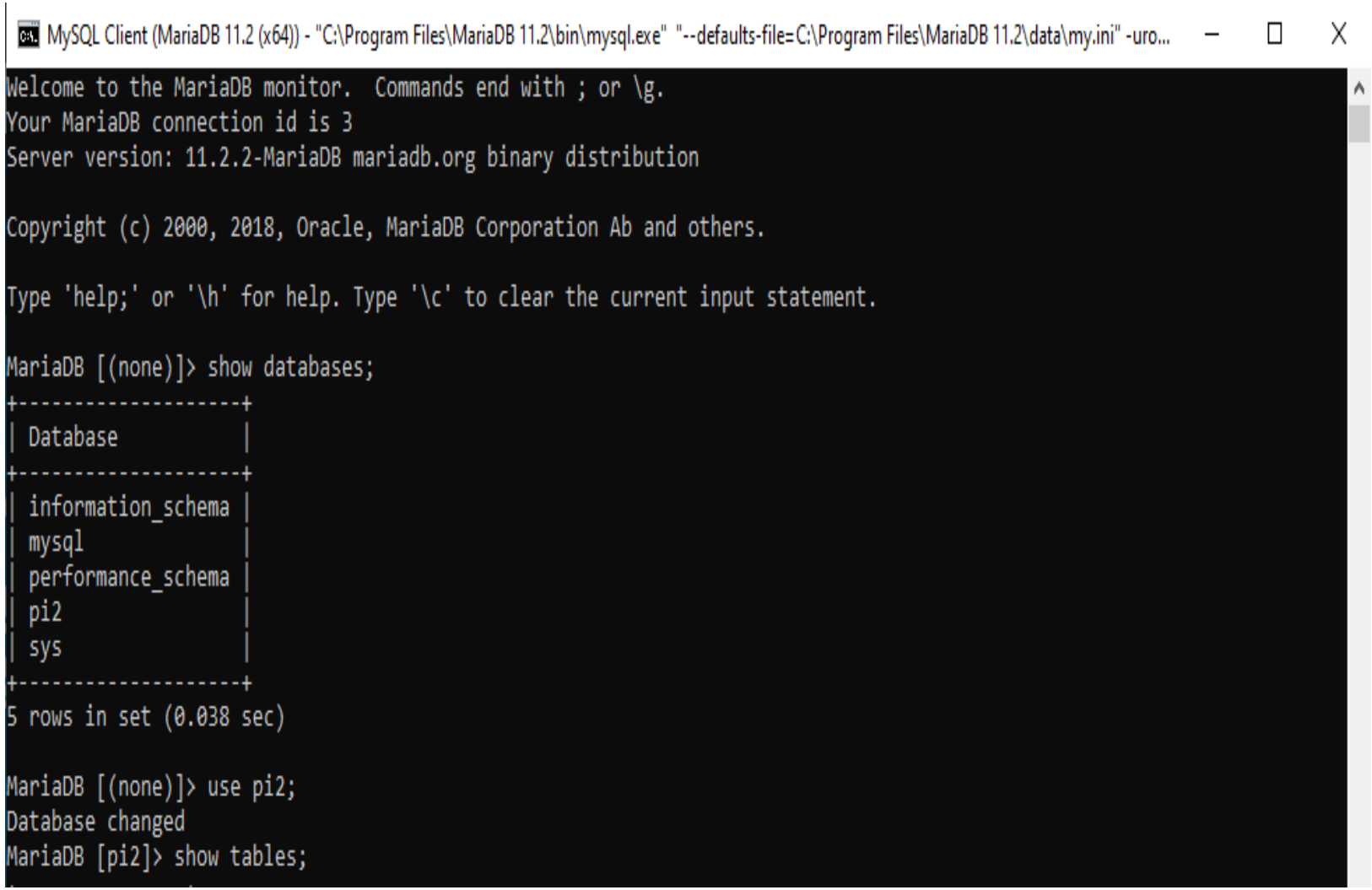
- Der Client ist rein textbasiert und erlaubt die Eingabe von SQL-Kommandos.
- Legen Sie einmalig eine Datenbank an:

```
> create database pi2;
```

- Dann wählen Sie die Datenbank aus:

```
> use pi2;
```

- Der Prompt zeigt an, welche Datenbank Sie gerade benutzen.



[ɛskjuːˈɛl] oder [ˈsiːkwəl];

Was ist SQL?

- Structured Query Language
- Datenbanksprache
- Deskriptiv (das gewünschte Ergebnis beschreibend)
- Basis: Relationenalgebra bzw. Mengenlehre
- Aufgaben
  - Definition relationaler Datenbanken (data definition language)
  - Bearbeitung und Auswertung der Daten (data manipulation language)

Merkmale von SQL

- Plattformunabhängigkeit
- Normung
  - American National Standards Institute (ANSI)
  - International Standards Organisation (ISO)
- Große Verbreitung
- Kleiner, aber mächtiger Sprachumfang

Anlegen von Tabellen

```
CREATE TABLE <dbname>.<tabellenname>  (
  <spaltenname> DATENTYP [NOT NULL] [{AUTO_INCREMENT | DEFAULT WERT}],
                ...
  <spaltenname> DATENTYP [NOT NULL] [{AUTO_INCREMENT | DEFAULT WERT}],
 [[CONSTRAINT <constraintname>] PRIMARY KEY (<spaltenname> [,… <sp.>]),]
 [[CONSTRAINT <constraintname>] FOREIGN KEY (<spaltenname> [, … <sp.>])
      REFERENCES <tabellenname> [(<spaltenname>, … <spaltenname>)]
         [ON {DELETE | UPDATE}
          {RESTRICT | CASCADE | SET NULL | SET DEFAULT | NO ACTION]},]
 [[CONSTRAINT <constraintname>] CHECK <bedingung>,]
 [[CONSTRAINT <constraintname>] UNIQUE (<spaltenname>, … <sp.>)]
 [COMMENT = '']
);
```

Datentypen

Datentyp	Inhalt	Beispiele
INTEGER	ganze Zahl	0 / -5 / 1500
REAL / DOUBLE	Fließkommazahlen	0.0 / 34.23 / -13.4534
NUMERIC(x,y)	Festkommazahlen	0.0 / 34.23 / -13.4534
TEXT	Text unbegrenzter Länge	„Lorem ipsum dolor…“
VARCHAR(max)	Zeichenkette variabler Länge	‘aA?‘ / ‘Test‘ / ‘‘
CHAR(fix)	Zeichenkette fixer Länge	‘aA?‘ / ‘Test‘ / ‘‘
DATE	Datumswert	2021-12-08
TIME	Tageszeit (ohne Datum)	08:47:51
TIMESTAMP	Datum mit Uhrzeit	2038-01-19 03:14:07
… WITH TIME ZONE	Zeitdaten mit Zeitzonendaten	08:47:51+01
BOOLEAN	Logischer Wert	TRUE / FALSE

Beispiel: Anlegen einer Tabelle für Kurse

```
MariaDB [pi2]> create table kurs (
    -> Modulnr varchar(6) not null,
    -> Modulname text not null,
    -> CrP integer default 6)
    -> ;
Query OK, 0 rows affected (0.028 sec)
MariaDB [pi2]> describe kurs;
+-----------+------------+------+-----+---------+-------+
| Field     | Type       | Null | Key | Default | Extra |
+-----------+------------+------+-----+---------+-------+
| Modulnr   | varchar(6) | NO   |     | NULL    |       |
| Modulname | text       | NO   |     | NULL    |       |
| CrP       | int(11)    | YES  |     | 6       |       |
+-----------+------------+------+-----+---------+-------+
3 rows in set (0.030 sec)
```

Tabelle mit drei Spalten
not null: darf nicht leer sein,
Default: wenn kein Wert angegeben wird, ist der Wert 6

Beschreibung der Tabelle kurse.

Aufgabe

Legen Sie eine Tabelle „Auto“ an, die drei Spalten hat:

- Nummer (INT)
- Marke (varchar(20))
- Modell (varchar(30))

Dabei dürfen die Einträge für Nummer nicht leer sein.

Lösung:

```
CREATE TABLE autos (
      Nummer INT NOT NULL,
      Marke VARCHAR(20),
      Modell VARCHAR(30)
     );
```

Kopieren, Löschen und Ändern von Tabellen

- Mit dem Kommando
  - `CREATE TABLE <dbname>.<tabellenname> AS <select-Anweisung>;`

  kann man das Ergebnis einer Abfrage (siehe Folie 149) als Tabelle anlegen.
- Mit dem Kommando
  - `DROP TABLE <dbname>.<tabellenname>;`

  wird eine Tabelle (samt Inhalt) gelöscht. Falls dabei die Referentielle Integrität verletzt würde, wird die Tabelle nicht gelöscht!
- Mit dem Kommando
  - `ALTER TABLE <dbname>.<tabellenname> …;`

  Kann man eine Tabelle ändern (z.B. Spalten hinzufügen oder löschen oder …)

  Beispiel: `ALTER TABLE autos ADD COLUMN farbe varchar(10) ;`

  Siehe: https://mariadb.com/kb/en/alter-table/

Aufgabe

Löschen Sie die Spalte `farbe` wieder.

Lösung:

Einfügen von Datensätzen

```
INSERT INTO <tabellenname> [(<spaltenname>, ...)]
   VALUES (<werteliste>)[,(<werteliste>),…,(<werteliste>)];
```

Achtung: Genauso viele Werte wie Spaltennamen!

Beispiele:

```
insert into kurs values ('INF1005', 'Praktische Informatik 1', 6);
insert into kurs (ModulNr, ModulName) values ('INF1005', 'Praktische Informatik 1‘);
```

Alternativ kann man auch das Ergebnis einer Abfrage einfügen:

```
INSERT INTO <tabellenname> [(<spaltenname>, ...)]
   AS <select befehl>;
```

Aufgabe

Fügen Sie einen VW Polo mit Nummer 1 und Farbe `blau` der Tabelle `autos` hinzu.

Lösung:

Löschen von Datensätzen

```
DELETE FROM <tabellenname>
    [WHERE <bedingung>];
```

Referentielle Integrität wird berücksichtigt.

```
> select * from kurs;
+---------+-------------------------+-----+
| ModulNr | ModulName               | CrP |
+---------+-------------------------+-----+
| INF1005 | Praktische Informatik 1 |   6 |
| INF5001 | Programmieren 1         |   6 |
+---------+-------------------------+-----+
> delete from kurs where ModulNr="INF1005";
Query OK, 1 row affected (0.006 sec)

MariaDB [pi2]> select * from kurs;
+---------+-----------------+-----+
| ModulNr | ModulName       | CrP |
+---------+-----------------+-----+
| INF5001 | Programmieren 1 |   6 |
+---------+-----------------+-----+
1 row in set (0.001 sec)
```

Ändern von Datensätzen

```
UPDATE <tabellenname>
    SET <spaltenname> = <ausdruck> [,<spaltenname> = <ausdruck> ...]
    [WHERE <bedingung>];
```

- Berechnungen in Ausdrücken möglich
- Berücksichtigung von Abhängigkeiten
- Sicherstellung der referentiellen Integrität

```
> update kurs set ModulNr="INF1001" where
ModulName="Programmieren 1";
Query OK, 1 row affected (0.007 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [pi2]> select * from kurs;
+---------+-----------------+-----+
| ModulNr | ModulName       | CrP |
+---------+-----------------+-----+
| INF1001 | Programmieren 1 |   6 |
+---------+-----------------+-----+
1 row in set (0.001 sec)
```

Datenabfrage

```
SELECT [DISTINCT] {      <tabellenname>.* |
           <tabellenname>.<attributname> [AS <alias>]|
                 <ausdruck> [AS <alias>] }
                 [,  ...]
   FROM <tabellenname> [<alias>]
        [LEFT JOIN | RIGHT JOIN | NATURALJOIN | JOIN] [USING | ON]
        [, ...]
   [WHERE <bedingung>]
   [GROUP BY {<tabellenname>.<attributname> | <alias>} [, ...]]
   [HAVING <bedingung>]
   [UNION <select befehl>]
   [ORDER BY {<tabellenname>.<attributname> | <alias>} [ASC|DESC]
          [, ...]]
;
```

Bestandteile des SELECTs

- **`SELECT`** (Selection)
  Auswahl der Attribute/Tabellenspalten
- **`DISTINCT`**
  Nur Anzeige unterschiedlicher Datensätze
- **`*`**
  Auswahl aller Attribute einer Tabelle
- **`AS`**
  Umbenennung eines Attributes / eines Ausdrucks
- **`<Ausdruck>`**
  Berechnungsvorschrift / Formel
- **`FROM … Kreuzprodukt / Join`**
  Auswahl der im Select-Befehl verwendeten Tabellen
- **`WHERE`** (Restriktion)
  Definition der Bedingungen zur Auswahl der Datensätze. Restriktion kann nur die Werte wahr oder falsch annehmen
- **`GROUP BY`**
  Spalten, nach denen gruppiert werden soll.
- **`HAVING`**
  Überprüfung der Eigenschaften einer Gruppe
- **`UNION`**
  Vereinigung der Ergebnisse zweier Select-Befehle
- **`ORDER BY`**
  Definition der Sortierung der Ergebnisdatensätze

Operatoren in WHERE-Klauseln

- `>; >= ; =; =<; <; <>`
- **`AND`**; `OR`; **`NOT`**
- **`IS (NOT) NULL`**: Überprüfung auf den Wert **`(NOT) NULL`**
- **`IN`**: Mengenoperator
- **`LIKE`**: Vergleichsoperator für Zeichenketten
  - `_`: Ein beliebiges Zeichen
  - `%`: beliebige Zeichenfolge
- **`BETWEEN`** … **`AND`** : Intervallangabe

Source - Befehl

- Mit dem source Befehl kann man SQL-Kommandos aus einer Datei einlesen und ausführen.
- Beispiel:

```
> source MeinOrdner/1_pi2_kurse_dozenten;
Database changed
Query OK, 0 rows affected (0.022 sec)
…
```

- Im Moodle-Kurs liegt die obige Beispiel-Datei. Nutzen Sie diese, wenn Sie die folgenden Beispiele nacharbeiten wollen.

Select – Varianten - 1

- Alle Kurse?

```
> select * from kurs;
+---------+--------------------------+------+-----------+
| ModulNr | ModulName                | CrP  | Dozent_id |
+---------+--------------------------+------+-----------+
| II1001  | Technische Informatik 1  |    6 |        13 |
| INF1003 | Programmieren 1          |    6 |        11 |
| INF1005 | Praktische Informatik 1  |    6 |        12 |
| TI5005  | Verteilte Algorithmen    |    6 |        12 |
+---------+--------------------------+------+-----------+
4 rows in set (0.001 sec)
```

- Alle Modulnummern?

```
> select ModulNr from kurs;
+---------+
| ModulNr |
+---------+
| INF1003 |
| INF1005 |
| TI5005  |
| II1001  |
+---------+
4 rows in set (0.001 sec)
```

Aufgabe

Selektieren Sie alle ModulNummern mit zugehörigen ModulNamen.

Lösung:

Select – Varianten - 2

- Alle Kurse von Dozent 13.

```
> select * from kurs where Dozent_id=13;
+---------+-------------------------+------+-----------+
| ModulNr | ModulName               | CrP  | Dozent_id |
+---------+-------------------------+------+-----------+
| II1001  | Technische Informatik 1 |    6 |        13 |
+---------+-------------------------+------+-----------+
1 rows in set (0.001 sec)
```

Aufgabe

- Selektieren Sie alle Kurse deren `ModulName` mit P beginnt.

Aufgabe

Selektieren Sie alle Zeilen, in denen der ModulName mit „P“ beginnt und die Modulnummer auf 3 endet.

Lösung: